

Java Scripts

Why do we need java Script?

Java Script make the web page interactive

JavaScript is a versatile programming language extensively used in web development. It empowers interactive features like form validation, dynamic content updates, and user interface enhancements. Furthermore, it's employed in server-side scripting, mobile app development, game development, and even desktop application development through frameworks like Electron.

How many data types are there in JavaScript ?

Primitive Data Types:

1. **Number**: Represents both integers and floating-point numbers (e.g., 42, 3.14).
2. **BigInt**: For integers larger than the Number type can handle (e.g., 123n).
3. **String**: Represents text (e.g., "Hello, World!").
4. **Boolean**: Represents logical values (true or false).
5. **Undefined**: A variable that has been declared but not assigned a value.
6. **Null**: Represents an intentional absence of value.
7. **Symbol**: Represents unique identifiers (introduced in ES6).

Non-Primitive Data Type:

8. **Object**: Used to store collections of data or more complex entities (e.g., { name: "John", age: 30 }). In form of key and value
9. **array**

In how many ways we can create variable?

Let, var, const.

Difference between let and var?

- **var**: Function-scoped. Variables declared with var are accessible throughout the function in which they are declared, regardless of block boundaries.
- **let**: Block-scoped. Variables declared with let are only accessible within the block {} in which they are defined.

Difference between global, block scope, functional scope?

1. Function scope is within the function.

Block scope is within curly brackets.

```
var character4 =
```

```
function foo() {  
  if(true) {  
    var character1 = "Robin"    //function scope  
    let character2 = "Ted"      //block scope  
    const character3 = "Barney" //block scope  
  }  
  console.log(character1) //Robin  
  console.log(character2) //not defined  
  console.log(character3). //not defined  
}
```

- **Global scope**: Variables declared outside of all functions are known as global variables and in the *global scope*. Global variables are accessible anywhere in the program.
- **Function scope**: Variables that are declared inside a function are called local variables and in the *function scope*. Local variables are accessible anywhere inside the function.

- **Block scope:** Variable that is declared inside a specific block & can't be accessed outside of that block. In order to access the variables of that specific block, we need to create an object for it.

What do mean by prompt?

`//##### WINDOW.prompt` : it means prompt is a template stored in window library.

Used to generate a dialog box for taking input .

```
const getPrompt= prompt("ENTER:",1);
console.log(typeof getPrompt); //string
```

what are the diifferent types of oprators in javaScript?

Arthmatic oprator-= +,-,*,/,%,--,++

2.ASSIGENMENT OPRATOR:+=, =, -=, *=, **=, /=, %= .

3.STRING OPRATPOR:

4. type of oprator:

5,.COMPARISSION OPRATORS : ==,===,!=,!==,<,>,>=,<=.

6.TERNARY OPRATOR

7.LOGICAL OPRATOR : && , || , !

Define array distruction?

//Array destructuring in JavaScript is a feature that allows you to extract multiple values from an

// array and assign them to distinct variables in a

// concise way. It simplifies the process of accessing and using array

// elements, improving code readability

// // and reducing verbosity. It does not modify the original array.

```
// const arr=["a","b","c","d"];
```

```
// const [data1,data2,data3,data4]=arr;//you can take any name to replace "data1"...
```

```
// console.log(data1,data2);
```

Difference between spread and rest operator?

The **rest operator** is used to collect multiple elements extra left into a single array or object.

The **spread operator** is used to expand (or "spread") elements of an array, or object, or iterable into individual elements

What do mean by hosting?

Hosting means to call anything like variable or function but,

It does not matter to call from top or below.

Ex:-in variable hosting is only work in var ,not in let and const

And

In function it work only in normal function call ,

Not work in function expression,arrow function&IIFE function.

What do mean by call back function?

To pass function as argument to a function parameter

Difference between parameter and argument?

Parameters

Parameters are **placeholders** defined in the function or method declaration.

They specify the type and number of inputs that the function can accept.

Parameters act as **local variables** within the function, and their scope is limited to the function itself.

Arguments

Arguments are the **actual values** or variables passed to a function when it is called. These values are assigned to the corresponding parameters during the function's execution.

Difference between slice and splice in array method?

Slice: this method is used to selected data in array and does not change the original array

splice: it is used to insert an hetrogenius type of data & delete the selected index data . so, it change the original array

Why do we use map method in js?

In JavaScript, the map() method is used to **transform an array** by applying a function to each element of the array and returning a **new array** with the transformed values. It is a powerful and concise way to perform operations on arrays without modifying the original array.

Difference between window and this?

window object:==

//it is a global object ,All things of javascript like function ,array etc...

// are kept inside this window object .

// this object:=====

// "this object" is a global object that refer to window object.

what do mean by promises?

A **Promise** in JavaScript is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value. It allows you to write asynchronous code in a more synchronous fashion, making it easier to read and maintain.

Key Principles of Promises

A Promise can be in one of three states:

- **Pending:** The initial state, neither fulfilled nor rejected.
- **Fulfilled:** The operation was completed successfully.
- **Rejected:** The operation failed.

When a Promise is created, it takes a function as an argument. This function is called the executor and it takes two arguments: resolve and reject.

The resolve function is called when the asynchronous operation is successful, and the reject function is called when it fails.

What is DOM?

// DOM stands for Document Object Model. It is a programming
// interface that represents the structure of a web page as a hierarchical
// tree of objects. This model allows programming languages like JavaScript
// to dynamically access, modify, and interact
// with the content, structure, and style of a web document.

// The DOM is essential for creating dynamic and interactive web applications.
// It bridges the gap between static HTML/CSS and dynamic JavaScript
functionality.

// For example, JavaScript can use the DOM to change HTML elements, update
styles,

// handle user interactions, and even create or remove elements in real-time.

// Key Features of the DOM----

// Tree Structure: The DOM represents the HTML document as a tree,
// where each element, attribute, and text is a node. For example,
// the <html> tag is the root node, and its children include <head> and <body>.
// Dynamic Interaction: It allows developers to update content,
// styles, and structure without reloading the page. For instance,
// you can change the text of a paragraph or add new elements dynamically.
// Event Handling: The DOM supports event handling, enabling
// actions like responding to button clicks or form submissions.
// Cross-Platform: It is platform-independent and works across different
// browsers and programming languages.

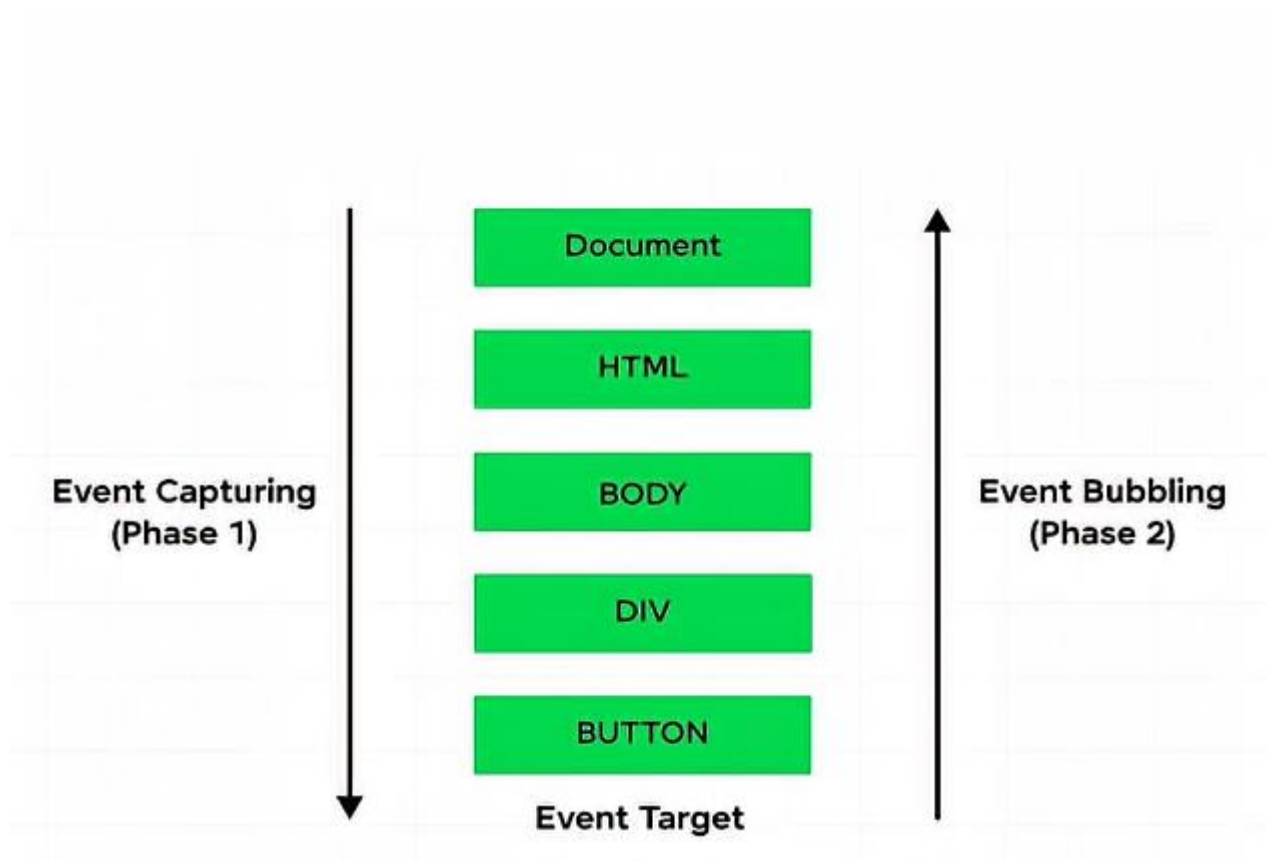
//as we know web page created using html elements
//whenever a web page is loaded in browser then, browser create dom.
//dom structure is created in form of tree of objects.

Difference between event bubbling and event capturing?

Event bubbling and event capturing are two distinct phases of event propagation in JavaScript. Here's a comparison of the two:

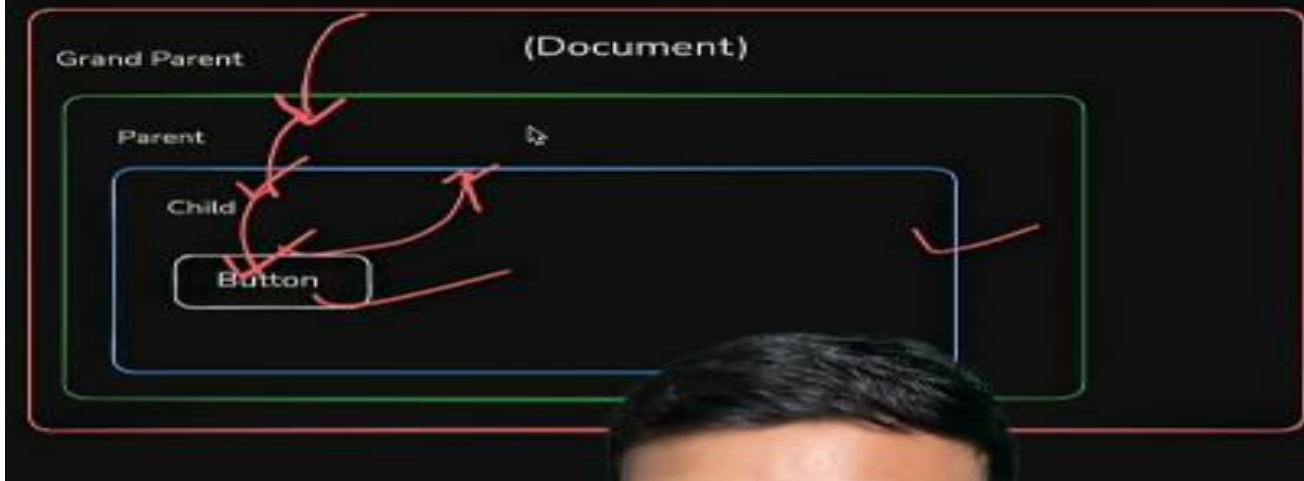
- **Event Bubbling:** This is the default behavior for most events in JavaScript. Events travel from the target element up through its ancestors to the root element. This phase is used for events that are expected to propagate through multiple levels of the DOM.
- **Event Capturing:** This phase is used when you want to prevent events from propagating through the DOM. You can enable capturing by using the capture option in addEventListener. This allows you to intercept events before they reach the target element.

- **Usage:** Use event bubbling for events that are expected to propagate through multiple levels of the DOM. Use event capturing when you need to control the order of event handling at specific levels.



Event Propagation

Capturing → Target → Bubbling



What are template literals in JavaScript?

Template literals in JavaScript are a feature that allows you to embed expressions, variables, and multi-line strings directly into a string, making your code more readable and flexible.

Variables

```
let a = 10;
```

```
let b = 20;
```

Expressions

```
${a+b}
```

String

```
`Sum of ${a} and ${b} is ${a+b}`
```

Q>Difference between let, var, and const. in javascript?

var	let	const
The scope of a <u>var</u> variable is functional or global scope.	The scope of a <u>let</u> variable is block scope.	The scope of a <u>const</u> variable is block scope.
It can be updated and re-declared in the same scope.	It can be updated but cannot be re-declared in the same scope.	It can neither be updated or re-declared in any scope.
It can be declared without initialization.	It can be declared without initialization.	It cannot be declared without initialization.
It can be accessed without initialization as its default value is "undefined".	It cannot be accessed without initialization otherwise it will give 'referenceError'.	It cannot be accessed without initialization, as it cannot be declared without initialization.
These variables are hoisted.	These variables are hoisted but stay in the temporal dead zone until the initialization.	These variables are hoisted but stays in the temporal dead zone until the initialization.

Q - Data types in JavaScript.?

DATA TYPES:= it refer to type of data variables i.e let var,const, can store.

> primitive data types : thay are immutable i.e they can not be modified.

variable,string,undefined,number,null,boolean,BigInt,

>non-primitive data types: thay are mutable i.e they can be modified.

array,object.

1.null:

```
var a=null;
```

```
console.log(typeof a); //object
```

2.boolean:

```
var a=true;
```

```
var b=false;
```

```
console.log( typeof a,typeof b);//boolean
```

3.BigInt:

```
var x=123;
```

```
const getbigint=BigInt(x);
```

```
console.log( getbigint);
```

```
console.log(typeof getbigint);
```

```
//other way to make a number big int?
```

```
var y=123n;
```

```
console.log( y);
```

```
console.log( typeof y);
```

```
=====
```

NON-PRIMITIVE DATA TYPES:==

4.array:

```
bug in array>
```

```
var a= [1,2,3,"dhruv"]
```

```
console.log(typeof a); //it will return object which is the error of js.
```

```
console.log(a[3]);
```

```
//how to check an array?
```

```
const checkArr = Array.isArray(a);  
console.log( checkArr);  
  
5.object:  
  
const a={  
  name:"dhruv",  
  email:"ronacksingh@gmail.com"  
}  
  
console.log(a);  
console.log( typeof a);  
console.log(a.name);
```

What is an array, and how to access its elements?

In **JavaScript**, an **array** is a special type of object used to store **multiple values in a single variable**.

Each value in an array is called an **element**, and each element has a **numeric index** starting from 0.

And in js you can store hetrogenious data in array.

You can create array using array syntax ex: `const arrName=[];`

Or by using array constructor ex; `const arr= new Array(10,12,13);`

You can acces array using:

Index: for specific element

For loop: itrarate hole array;

ForEach:

```
fruits. forEach(fruit => console.log(fruit));
```

- Purpose of the isNaN function.?

In JavaScript, NaN stands for "Not-a-Number". The isNaN() function is used to determine whether a value is NaN or not. This function is part of the global object and is widely supported across all browsers.

Usage and Syntax

The syntax for isNaN() is straightforward:

isNaN(value)

- **value**: The value to be tested.

The function returns a boolean value: true if the value is NaN, otherwise false.

How it Works

When you pass a value to isNaN(), it first converts the value to a number if necessary. This coercion can sometimes lead to surprising results. For instance:

```
console.log(isNaN('Hello')); // true
```

```
console.log(isNaN('123')); // false
```

```
console.log(isNaN(true)); // false
```

```
console.log(isNaN(false)); // false
```

What is null vs undefined?

Both indicate the absence of the value.

NULL: it is used when we don't want to give value to variable

Undefined: In this variable is declared but not provided value which returns undefined.

What do you understand by Closures in JavaScript?

In JavaScript a closure is a function that remembers the environment in which it was created even after the outer function has finished executing

In below code:

With the use of closure function (i.e counter) you can access private variable (i.e count) outside function.

```
function counter( )  
  
  let count=0;  
  
  return{  
  
    increment: function(){  
  
      count++;  
  
      return count;  
  
    },  
  
    displayCount: function(){  
  
      let message = "Current count: " + count;  
  
      return message;  
  
    }  
  
  }  
  
  const myCounter = counter();
```

```
console.log(myCounter.displayCount());  
console.log(myCounter.increment());  
console.log(myCounter.displayCount( )):
```

How do setTimeout and setInterval work?

setTimeout and setInterval ??

<https://www.youtube.com/watch?v=MX48mv73jf8&t=1812s>

In JavaScript, setTimeout and setInterval are functions used to schedule tasks to run after a specified amount of time.

They're both part of the Web APIs provided by the browser (or Node.js in a server environment), and they allow

you to create delays or repeated actions in your code.

setTimeout()

Used to execute a function once after a specified delay

setInterval()

Used to execute a function repeatedly at specified intervals

Explain the concept of Promises in JavaScript. ??

[click to go to video](#)

Promises handle asynchronous tasks in JavaScript by providing a more readable and structured approach than callbacks for handling outputs

- **Pending** → Initial state, neither fulfilled nor rejected.
- **Fulfilled** → Operation completed successfully, and a value is available.
- **Rejected** → Operation failed, and a reason (error) is available.

```
• // const data ={name: "John", age: 30 };
•
• function fetchData(){
•
•   return new Promise((resolve, reject) => {
•     setTimeout(() => {
•       resolve(data);
•     }, 1000); // 2 sconds
•
•   })
• }
•
• fetchData().then(
•   data => {
•     console.log("Data: ", data);
•   }).catch(err => {
•     console.log("Error: ", err);
•   })
• }
```

What is Event Delegation?

[video for event deligation](#) <- you tube video refrence

A technique that allows you to manage events for multiple elements more efficiently by using a single event listener on a common parent element rather than on each individual child element

```
<div className="box">
```

```
<ul id="itemList">
```

```
<li>Item 1</li>
```

```
<li>Item 2</li>
```

```
<li>Item 3</li>
```

```
</ul>
```

```
</div>
```

```
<script>
```

```
//without Event diliation !
```

```
// const items = document.querySelectorAll('#itemList li');
```

```
// items.forEach(item => {
```

```
// item.addEventListener("click", function () {
```

```
// console.log("Clicked: ", this.textContent);
```

```
// })
```

```
// })
```

```
//with Event diliation !
```

```
const items = document.getElementById('itemList');
```

```
items.addEventListener("click", function(e){
```

```
if(e.target.tagName === "LI"){
```

```
console.log("Clicked: ", e.target.textContent);
```

```
}
```

```
})
```

What is the difference between a promise and an async/await function?

async/await

async/await is a cleaner, more readable way to handle Promises

Inside these functions, you use await to pause execution until the Promise resolves

promise

promises are the underlying mechanism

Promises are the building blocks that async/await works on top of, but Promises themselves don't "enclose"

async/await

What are Weak maps and Weak sets in JavaScript?

Both WeakMap and WeakSet are non-iterable, have no size property, and are ideal for temporary data

storage and memory-efficient management of objects.

```
//weak map:
```

```
// let weakMap = new WeakMap();
```

```
// let obj = { name: "Adam" };
```

```
// weakMap.set(obj, "Employee");
```

```
// // obj = null; //use to remove data in obj!
```

```
// console.log(weakMap.get(obj));
```

```
//weak set:
```

```
let weakSet = new WeakSet();
```

```
let obj = { name: "Adam" };
```

```
weakSet.add(obj, "Employee");
```

```
// obj = null; //use to remove data in obj!
```

```
console.log(weakSet.has(obj));
```

How does JavaScript handle memory management?

JavaScript allocates memory when variables, objects, or functions are created. Primitive data types like numbers and strings are stored in **stack memory**, which is fast and automatically freed when variables go out of scope. Complex data types like objects and arrays are stored in **heap memory**, where memory is dynamically allocated during runtime. References to these objects are stored in the stack.

Describe the difference between Shallow and Deep copying ??

Shallow copying

In the shallow copy of an object, the main properties are copied, but any nested objects or arrays are still

linked to the original

Shallow copying



Tree

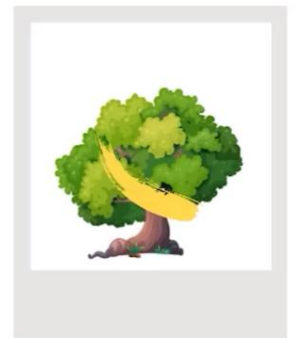


Photo of the tree

Deep copying

With a deep copy, every part of the object, including all nested objects or arrays, is fully copied, ensuring the

modifications to the copy doesn't affect the original

ex:

```
// Shallow Copy
```

```
// const original = { name: "John", address: { city:
```

```
// "Delhi" } };
```

```
// //way 1:Object.assign
```

```
// const shallowCopy = Object.assign({}, original);
```

```
// // way 2: Spread Operator
```

```
// const shallowCopy2 = { ... original};
```

```
// shallowCopy2.address.city = "New city";
```

```
// console.log(original.address.city);
```

```
//deep copy
```

```
// const original = { name: "John", address: { city:
```

```
// "Delhi" } };
```

```
// const deepCopy = JSON.parse(JSON.stringify(original)); //it will convert the  
object into the string
```

```
// deepCopy.address.city="new city";
```

```
// console.log(deepCopy.address.city);
```

```
// console.log(original.address.city);
```

Explain the Observer pattern and how it relates to JavaScript??

The Observer Pattern is a design pattern in which an object (called the subject) maintains a list of its dependents

(called observers) and notifies them of any changes, typically through a broadcast mechanism

```
const button = document.querySelector("button");  
💡  
function observer1(){  
  console.log("Observer 1 notified");  
}  
  
function observer2(){  
  console.log("Observer 2 notified");  
}  
  
button.addEventListener("click", observer1)  
button.addEventListener("click", observer2)
```

Explain the **Observer pattern** and how it relates to JavaScript



Diff b/w if else & ternary operator?

If else : it is statement e it is not evaluated on a single value.

ternary operator: is a expression i.e it is evaluated in single value and we can store it in variable.

Ex:- `const a= condition? "true" : "false";`

Is JavaScript Single-Threaded or Multi-Threaded ?

JavaScript is fundamentally **single-threaded**, meaning it executes one operation at a time within a single thread. This design ensures that only one piece of code runs at any given moment, simplifying how developers reason about code execution. However, JavaScript's runtime environment (e.g., browsers or Node.js) enables it to handle asynchronous tasks efficiently, giving the illusion of multitasking.

How Single-Threading Works in JavaScript

JavaScript operates using a **call stack**, which follows a Last In, First Out (LIFO) principle. Functions are pushed onto the stack when called and popped off once completed. This ensures that only one function executes at a time. For example:

```
function first() {
```

```
  console.log("First function");
```

```
}
```

```
function second() {
```

```
  console.log("Second function");
```

```
}
```

```
first();
```

```
second();
```

The output will always be:

First function

Second function

This sequential execution is a hallmark of single-threaded behavior .

[JavaScript Event Loop?](#)

The **event loop** is the core mechanism that allows JavaScript, despite being **single-threaded**, to handle **asynchronous operations** without blocking the main thread. It coordinates between the **call stack**, **Web APIs**, and **task**

queues to ensure smooth execution of both synchronous and asynchronous code.

JavaScript executes code **synchronously** by default, but asynchronous tasks like `setTimeout`, `fetch`, and **Promises** are handled outside the call stack by Web APIs, then queued for execution when the stack is clear.

Key Components:

- **Call Stack:** Executes functions in a **LIFO** order.
- **Web APIs:** Handle async tasks like timers, HTTP requests, and DOM events.
- **Callback (Task) Queue:** Holds macrotasks (`setTimeout`, `setInterval`, I/O callbacks).
- **Microtask Queue:** Holds microtasks (`Promise.then`, `MutationObserver`) which run **before** macrotasks.
- **Event Loop:** Continuously checks if the call stack is empty, then moves tasks from queues to the stack.

Example:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("setTimeout Callback");  
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log("Promise Resolved");  
});
```

```
console.log("End");
```

Output:

Start

End

Promise Resolved

setTimeout Callback

Explanation:

1. `console.log("Start")` → runs immediately.
2. `setTimeout` → callback goes to **task queue**.
3. `Promise.then` → callback goes to **microtask queue**.
4. `console.log("End")` → runs.
5. Event loop processes **microtasks first**, so "Promise Resolved" appears before "setTimeout Callback".

CODE OF HTML AND JAVA- SCRIPT

```
// console.log("hello world");  
// console.warn("this is used for warning...");  
// console.error("this is used for error .");  
//## typeof :is used for finding the datatype .  
// console.log( typeof "length:");  
// console.log( typeof "1234");  
// console.log( typeof 1234);  
// console.log( "1234 5678".length);  
//isNaN function: it stands for not a number.  
//is use to check the value is number or not  
// var a= 123;  
// console.log(isNaN("123")); //false  
// console.log(isNaN("string")); //true  
  
//#### TRIM:
```

// console.log("length:", " dhruv singh " .length);//it used to remove space before & after name.

// console.log("length:", " dhruv singh " .trim().length);

// console.log("length:", " dhruv singh " .trimStart().length);//used to remove strteing space

// console.log("length:", " dhruv singh " .trimEnd().length);

//### quates:

// console.log("hello 'dhruv' ");

//console.log('hello "dhruv" ');

// ## brackets:

// 1.parathes brackets: ().

// 2.angle brackets: <>.

// 3.curly brackets: {}.

// 4.square/array brackets: [].

// ## VARIABLES: VAR, LET, CONST.

//### SCOPE:--

// var a= 1;

// const b=2;

// let c= 3;

// console.log(a);

// console.log(b);

// console.log(c);

// 1.global scope:

// var a= 1;

// const b=2;

```
// let c= 3;
// console.log(a);
// console.log(b);
// console.log(c);
// {
// console.log(a);
// console.log(b);
// console.log(c);
// }
//block scope:
// {
// var a= 1;
// const b=2;
// let c= 3;
// console.log(a);
// console.log(b);
// console.log(c);
// }
```

// # let, const cant be ascess from block scope :- THIS EXAMPLE PROVEES
THAT in above ex.

```
// {var a= 1;
// const b=2;
// let c= 4;
// }
// console.log(a);
```

```
// console.log(b);
// console.log(c);
// //####call using FUNCTION:---
//golabal function call:
// var a= 1;
// const b=2;
// let c= 3;
// function abc1() {
// console.log(a);
// console.log(b);
// console.log(c);
// }
// abc1();
// // LOCAL function call:
// {var x=123;
// const cc=111;
// } //block scope

// let y=124;
// function abc() {
// var a= 1;
// const b=22;
// let c= 3;

// console.log(a);
```

```
// console.log(b);
// console.log(c);
// console.log(x);
// console.log(y);
// // console.log(cc);

// }
// abc();
//#### DECLARATION:--- & UPDATION
// 1.var: var can be re-declare
/*
var a=1;
var a=2;
console.log(a);*/
// 2.let: it is not re-declare but,can be updated
// let a=1;
// a=2;
// console.log(a);
// 3.const: CAN NOT BE UPDATED/or re-declare

// const a=1;
// const a=2;
// console.log(a);
//##### concatenation:____
// const a="dhruv";
```

```
// const b= "singh";  
// console.log(a+b);      ////OUTPUT: dhruvsingh  
// console.log(a+" "+b);   //// OUTPUT: dhruv singh
```

```
//#####_____TEMPLATE LITRAL STRING:--
```

```
// const a="dhruv";
```

```
// const b= "singh";
```

```
// console.log(`${a} ${b}`); ////to create space
```

// //##### WINDOW.prompt : it means prompt is a template stored in window library.

//it is used to genrate a dialog box.

```
// const getPrompt= prompt("ENTER:",1);
```

```
// console.log(typeof getPrompt); //string
```

```
//=====
```

// ##### DATA TYPES:= it refer to type of data variables i.e let var,const, can store.

```
// **** primitive data types
```

:variable,string,undefined,number,null,boolean,BigInt,

```
// **** non-primitive data types: array,object.
```

```
// 1.null:
```

```
// var a=null;
```

```
// console.log(typeof a); //object
```

```
// 2.boolean:
```

```
// var a=true;
```

```
// var b=false;
```

```
// console.log( typeof a,typeof b);//boolean
```

```
// 3.BigInt:
```

```
// var x=123;

// const getbigint=BigInt(x);

// console.log( getbigint);

// console.log(typeof getbigint);

// //other way to make a number big int?

// var y=123n;

// console.log( y);

// console.log( typeof y);

// =====

// ##### NON-PRIMITIVE DATA TYPES:==

// 4.array:

// bug in array>

// var a= [1,2,3,"dhruv"]

// console.log(typeof a); //it will return object which is the error of js.

// console.log(a[3]);

// //how to check an array?

// const checkArr =Array.isArray(a);

// console.log( checkArr);

// 5.object:

// const a={

//   name:"dhruv",

//   email:"ronacksingh@gmail.com"

// }

// console.log(a);

// console.log( typeof a);
```

```

// console.log(a.name);

// 6.FUNCTION:

// function fun()

// {

//   console.log("hello from functions...");

//   return "Data";

// }

// fun();

// console.log( typeof fun );//function

// const getfun=fun();

// // console.log(getfun);

// console.log( typeof getfun);//string

//=====

// var a=1;

// var b=2;

// console.log("a=",a);

// console.log("b=",b);

//=====

// var a=1;

// var b=2;

// a++;

// console.log("a=",a);

// console.log("b=",b);

```

```
//=====

//push,pop,unshift, and shift in array

// //push: will store data at last of array

// var a=["a","b","c"];

// console.log("before:",a);

// a.push("d");

// console.log("after:",a);

// //pop:will remove element from the last

// var a=["a","b","c"];

// console.log("before:",a);

//a.pop()

// a.pop("c");

// console.log("after:",a);

// // //shift : it will remove from starteing

// var a=["a","b","c"];

// console.log("before:",a);

// a.shift();

// console.log("after:",a);

// //unshift: add data in starteing

// var a=["a","b","c"];

// console.log("before:",a);

// a.unshift("x");

// console.log("after:",a);

//##### operator:---
```

//AROMATIC OPERATOR:+,-,*,**,--,++,/,%.

// a=2;

// b=3;

// console.log(a**b);

//2.ASSIGNMENT OPERATOR:+=, =, -=, *=, **=, /=, %= .

// a=12;

// a+=2; //a= a+12

// console.log(a)

//3.STRING OPERATOR:

// var a="hello";

// var b=" world";

// a+=b;

// console.log(a);

// 4.type of operator:

// var a="hello";

// var b=1;

// console.log(typeof a, typeof b);

// 5. COMPARISON OPERATORS : ==,===,!=,!==,<,>,>=<=.

// var a= 2;

// var b= "2";

// console.log(a==b); // check only value/not its type

// var a= 2;

// var b= "2";

// console.log(a===b); //check both value & its type.

//!= &&& !==

```

// var a= 2;

// var b= 3;

// console.log( a!=b);

// var a= 2;

// var b= "2";

// console.log( a!==b);

// //> &&& <

// var a= 2;

// var b= 3;

// console.log( a>b);

// //>=

// var a= 2;

// var b= 2;

// console.log( a>=b);

//6.TERNARY OPRATOR:--

// //YOURcondiition? "true": "false"

// var a=5;

// var b= 3;

// console.log(a>b? "a is garter then b" : "a is not grater than b");

// 7.LOGICAL OPRATOR : && , ||,!.

//&&: both statement should be true....

// var a= 2;

// var b= 3;

// console.log( a==3 && b==3);

// console.log( a==2 && b==3);

```

```

// || : one from the two statements should be true..

// var a= 2;

// var b= 3;

// console.log( a==3 || b==3);

// // !:

// var a= true;

// console.log(!a);

//=====

// if-else, else-if ,nested if.


//while and do while loop , for loop..

// var i= 1;

// while(i<=10)

// {

//   console.log("2 *",i,"=",2*i);

//   i++;

// }

// do while loop---

// do{

//   console.log("2 *",i,"=",2*i);

//   i++;

// } while(i<=10);

// ##### array : for in and for of-

//for in:=

// const arr=["a","b","c"];

```

```
// for(arrindex in arr){    // array index is obj name
//   console.log(arrindex);
// }
// for of:
// const arr2=["a","b","c"];
// for(arrdata of arr){      //array data is obj name
//   console.log(arrdata);
// }
```

//Q > CLONE PROBLEM IN THIS ..

// both a and b have same value??

```
// const a=["a","b","c"];
```

```
// var b= a;
```

```
// a.push("d");
```

```
// console.log(a);
```

```
// console.log(b);
```

// //solution 1...: for small kind of array datas.

```
// const a=["a","b","c"];
```

```
// var b= ["a","b","c"];
```

```
// a.push("d");
```

```
// console.log(a);
```

```
// console.log(b);
```

// //solution 2...

```
// const a=["a","b","c"];
```

```
// var b= [].concat(a);
```

```
// a.push("d");
```

```

// console.log(a);
// console.log(b);
//solution 3...
// spread(...)method.
// const a=["a","b","c"]
// console.log(...a);
// const arr=[...a];
// // const obj={...a};
// console.log(arr);
//solution 4:-
// const a=["a","b","c"];
// var b= [...a];
// a.push("d");
// console.log(a);
// console.log(b);
// ##### SLICE OPRATOR:
// const a=["a","b","c"];
// var b= a.slice(0);
// a.push("d");
// console.log(a);
// console.log(b);
//+=====
// const a= ["a","b","c","d"];
// const sliceresult= a.slice(1,2);
// console.log(sliceresult);//b

```

```
//### for loop with array
// const a=["a","b","x"];
// for(i=0;i<=a.length;i++)
// {
//   //array index
//   console.log(i);
//   // //array data
//   console.log(a[i]);
// }
//=====
```

//### array DESTRUCTION :

//Array destructuring in JavaScript is a feature that allows you to extract multiple values from an

```
// array and assign them to distinct variables in a
// concise way. It simplifies the process of accessing and using array
// elements, improving code readability
// // and reducing verbosity. It does not modify the original array.
// const arr=["a","b","c","d"];
```

// const [data1,data2,data3,data4]=arr; //you can take any name to replace "data1".

```
// console.log(data1,data2);
//##### object destrucuting:---
// const arr = {
//   sname:"dhruv",
//   semail:"ronacksingh@gmail.com",
//   snumber:9911
// }
```

```
// console.log(arr);
// const {sname,semail,snumber}=arr;
// console.log(sname,semail);
// RENAME THE OBJECT IN DSETRUCTING ----
// const arr = {
//   sname:"dhruv",
//   semail:"ronacksingh@gmail.com",
//   snumber:9911
// }
// // console.log(arr);
// const {sname:studentname,semail,snumber}=arr;
// // console.log(sname,semail); // it will genrate error now
// console.log(studentname,semail);
//##### array of objects ---->
```

```
// const arr =[
//   {sname:"dhruv",
//     semail:"ronacksingh@gmail.com",
//     snumber:9911}
//   ,
//   {sname:"pritish",
//     semail:"ronacksingh@gmail.com",
//     snumber:9911}
//   ,
//   {sname:"vaibhav",
```

```

//      semail:"ronacksingh@gmail.com",
//      snumber:9911}
//]

// asccess the hole array-
// console.log(arr);

//access the paticular object in array-
// const [obj1,obj2,obj3]=arr;
// console.log(obj1);

//=====

//##### nested DISTRUCTEING


// const arr =[
//   {sname:"dhruv",
//     semail:"ronacksingh1@gmail.com",
//     snumber:9911}
//   ,
//   {sname:"pritish",
//     semail:"ronacksingh2@gmail.com",
//     snumber:9911}
//   ,
//   {sname:"vaibhav",
//     semail:"ronacksingh3@gmail.com",
//     snumber:9911}
// ]

//how to get paticular object data?

```

```
//way1 :
```

```
// // const [obj1,obj2]=arr;
```

```
// console.log(obj1.sname);
```

```
//way 2:
```

```
// const [{sname},obj2]=arr;
```

```
// console.log(sname);
```

```
//way 3: for multiple sname
```

```
// const [{sname},{sname}]=arr;//this line causes error..
```

```
// const [{sname},{sname:sname2}]=arr; //CHANGE SECOND OBJECT  
sname which is a key to sname2
```

```
// console.log(sname,sname2);
```

```
//=====
```

```
//ARRAY :-
```

```
//update:
```

```
// const arr=["a","b","c","d"];
```

```
// arr[0]=apple;
```

```
// //insert:
```

```
// const arr=["a","b","c","d"];
```

```
// arr.push("e");
```

```
// console.log(arr);
```

```
//filter
```

```
//=====
```

```
//object:
```

```
// const objdata={
```

```
//  course:"HTMLAllCollection",
//  name:"dhruv",
// }
// console.log(objdata.course);
// //way1:get
// const getcourse=objdata.course;
// console.log(getcourse);
// //way2:
// const getcourse2=objdata['course'];
// console.log(getcourse2);
//update:
// //way1:
// objdata.course="javaa";
// console.log(objdata);
// way 2:
// objdata['course']="javaas";
// console.log(objdata);
// //delete:
//way1:
// delete objdata.name;
// console.log(objdata);
//way2:
// delete objdata['name'];
// console.log(objdata);
//=====
```

```

//iterate object:--
// const objdata={
//   course:"HTMLAllCollection",
//   name:"dhruv",
// }
// for( key in objdata)
// { //this display the object key..
//   //console.log(key);
//   //this display the object data values..
//   console.log(objdata[key]);
//   // console.log(objdata.key)//undefined
// }

// =====

// //object.keys:==
// const objdata={
//   course:"HTMLAllCollection",
//   name:"dhruv",
// }
// // findings key through "object.key".
// const getKeyResults=Object.keys(objdata);
// console.log(getKeyResults);
// =====
//object.values:
// finding object values ...
// const getKeyResults=Object.values(objdata);

```

```
// console.log(getkeyResults);

//=====

//computed properties: it refer to pass variable as key & value in object.

//problem-

// var key1="name";
// var key2="sr.name";
// var value1="dhruv";
// var value2="singh";
// const objnew={};
// obj.key1=value1;
// obj.key2=value2;
// console.log(objnew);
//above line of code not work .

//solution-

/// using compute property now
// var key1="name";
// var key2="sr.name";
// var value1="dhruv";
// var value2="singh";
// const objx={
//   [key1]:value1,
//   [key2]:value2
// }
// console.log(objx);
```

```
//=====
```

```
//function:--
```

```
//declaration=
```

```
// function funName(){
```

```
//   console.log("fun.workeing");
```

```
// }
```

```
// funName();
```

```
////////////////////////////////////
```

```
//function expression:==
```

```
// Not Hoisted – You must define the function before calling it.
```

```
// Can be Anonymous or Named – Named expressions help in debugging.
```

```
// First-Class Functions – Can be stored in variables, passed as arguments,  
or returned from other functions.
```

```
// Useful for Callbacks – Commonly used in event handlers, timers, and  
array methods.
```

```
//when we call a function using a variable name.
```

```
//types --
```

```
//anonymous function: a function does not have a name..
```

```
// const a =function(){
```

```

// console.log("wokring");

// }

// a(); ////we have to call function expression

// ARROW FUNCTION:

// const a =()=>{

//   console.log(" arrow fun .wokring here");

// }

// // a();

// console.log(a());//undefined


// const funobj= ()=>>

//   "without using curly brackets ..";

// console.log(funobj());

// // funobj();//no output


// IIFE : imediatly invoke function.

// (

//   function funName(){

//     console.log("fun.workeing");

//   })

// ()//called function.we have to call iife .

// // ARGUMENTS & PARAMETERS:==\

// function funName(a,b) // here a & b are the parametrs.

// {

//   console.log("adding a & b=",a+b);

```

```
// }  
  
// funName(1,2);// 1&2 are arguments.  
  
//=====
```

// ## function expression using arguments & parameters:

```
// const myobj =function(a,b){  
// console.log("wokring");  
// }  
  
// myobj(1,2);
```

// ## function arrow using arguments & parameters:

```
// const myobj =(a,b)=>{  
// console.log("wokring");  
// }  
  
// myobj(1,2);
```

// ## IIFE using arguments & parameters:

```
// (function(a,b){  
// console.log("wokring");  
// })  
  
// (1,2);
```

```
//=====
```

// RETURN IN function..

```
// function fun(){  
// return("hello");  
// }  
  
// fun()  
  
// console.log(fun());
```

```
//=====
```

```
// default parameter:
```

```
//means addeing default value to the parameter.
```

```
//problem??
```

```
// function ab(a,b){
```

```
//   console.log("values of a & b:",a,b);//1,undefined
```

```
//   console.log("a+b:",a+b);//NAN
```

```
// }
```

```
// ab(1);
```

```
//solution:using default parameter
```

```
// function ab(a,b=2){
```

```
//   console.log("values of a & b:",a,b);
```

```
//   console.log("a+b:",a+b);
```

```
// }
```

```
// ab(1,1);
```

```
//=====
```

```
// parameter destructuring:
```

```
//problem:
```

```
// const objdata={
```

```
//   name:"dhruv",
```

```
//   roll_no:123,
```

```
//   class:1,
```

```
// }
```

```
// function myfun(getdata){
```

```
//   //this will lead to more time complexity!!.
```

```
// console.log(getdata);  
// console.log(getdata.name);  
// console.log(getdata.roll_no);
```

```
// }
```

```
// myfun(objdata);
```

```
//solution: using paramter destructing:
```

```
// const objdata={
```

```
//   name:"dhruv",
```

```
//   roll_no:123,
```

```
//   clas:12,
```

```
// }
```

```
// function myfun({name,roll_no,clas})
```

```
// { // this will reduce the time compleity
```

```
//   console.log(name,clas);
```

```
// }
```

```
// myfun(objdata);
```

```
//=====
```

```
=====
```

//rest oprator : it gives the rest data left in form of Array /array of object.

// syntax: ...AnyName

//you can take any name in place of c .ex:...restopractor,,etc.

```
// function myfun(...c)
```

```
// {console.log(c);}
```

```
// myfun(1,2,3,4,);
```

```

// // its use???
// function myfun(a,b,...restValues)
// {console.log("a&B:",a,b);
//     console.log( "rest numbers1,2,:",restValues);}
// myfun(3,4,2,222,333);
// //
//=====

//HOSTEING:---
//means to call the variable or a function ,,but it does not mean to call
//from below or the top.
//* hosting in function declaration ====>
// yes,it work in function declaration.ex:
// fun();
// function fun(){
//     console.log("hello ...");
// }
//* hosting in function expression:====>
// doesnt work in function expression.
// obj();
// const obj = function(){
//     console.log("hello");
// }
// * hosting in arrow function=====>
// // does not work in arrow function..
// obj();

```

```
//  const obj =()=>>{  
//  console.log("hello");  
// }  
//iife function: not work in it!  
// ()  
// (  
// function fun(){  
//  console.log("hello ...");  
// })
```

```
// // hosting in variable : =====>
```

```
// //yes its work in variable .ex:
```

```
// // console.log(a);
```

```
// var a=1;
```

```
// hosting in let : =====>
```

```
// //no it does not work in let .ex:
```

```
// console.log(a);
```

```
// let a=1;
```

```
// hosting in const : =====>
```

```
// //no it does not work in const .ex:
```

```
// console.log(a);
```

```
// const a=1;
```

```
//=====
```

```
=====
```

```
// function inside function:--
```

```
// to declare function inside the function.
// function parent(){
//   console.log("i am the parent fun.");
//   function child(){
//     console.log("i am the child one fun...");}
// child();
// }
// parent();
//=====
=====
```

//function return function:-

// by using return in this all child data is stored in the parent..

```
// function parent(){
//   console.log("i am the parent fun.");
//   function child(){
//     console.log("i am the child one fun...");}
//   return child;
//   }
//   parent();
//   console.log(parent());
```

//to display the child data stored in parent we use this"()" -

// parent() ();

```
//=====
=====
```

// //call back function :- pass function inside function

```
// function fun1(){
```

```
// console.log("hello i am the fun 1..");  
// }
```

```
// function fun2(obj){  
// console.log("hello i am the fun 2..");  
// fun1();  
// }
```

```
// fun2(fun1);
```

```
// // way2:
```

```
// function fun2(obj){  
// console.log("hello i am the fun 2..");  
// obj();  
// }
```

```
// fun2(fun1);
```

```
//=====
```

```
=====
```

```
// HIGHT ORDER FUNCTIONS:====every function in it has its own quality
```

```
//1.forEach method:use to display array content
```

```
// const fruits=[1,2,3,4,5];
```

```
// fruits. forEach(fruit => console.log(fruit));
```

```
// const arrdata=[1,2,3,4,5];
```

```
// arrdata.forEach( (item,index,arr)=>{
```

```
// console.log(item,index,arr);
```

```
// }
```

```
// )
```

```

// things we can do with forEach:
// const arrdata=[1,2,3,4,5];
// arrdata.forEach( (item,index,arr)=>{
//   console.log(item*2);
// }
// )
// console.log("original:",arrdata);
//return does not work in forEach:ex-
// const arrdata=[1,2,3,4,5];
// const getitem= arrdata.forEach( (item)=>{
//   return item;
// }
// )
// console.log(getitem);
//=====

// / 2. map :- return a new array.. used to give function to array data
// const arrdata=[1,2,3,4,5];
// const getitem= arrdata.map( (item,index,arr)=>{
//   return item *2;
// }
// )
// console.log("result array:",getitem);
// //no changes made to original array....
// console.log("original array:",arrdata);
//=====

```

```
// // 3. filter :--- used to get specific data
```

```
// const arrdata=[1,2,3,4,5];
```

```
// const getitem= arrdata.filter( (item,index,arr)=>{
```

```
//   return item % 2== 0;
```

```
// // return item==2; //[2]
```

```
// }
```

```
// )
```

```
// console.log("result array even numbers:",getitem);
```

```
// //no changes made to original array....
```

```
// console.log("original array:",arrdata);
```

```
//4. reduce :_---
```

```
// it takes two parameter one is call by function and other is enetialisation
```

```
// within a call by function it contains two parameter one is acumilatore
```

and

```
// second one is current value and returns a value..
```

```
// //note: acumilater = initialize value + current value.
```

```
//with the use of it we add all the value in array
```

```
// var arr=[1,2,3,4,5];
```

```
// const result=arr.reduce((acu,curvalue)=>{
```

```
// console.log("acu value:", acu);
```

```
// console.log("curvalue",curvalue);
```

```
// return acu + curvalue;
```

```
// },0)
```

```
// console.log("sum of arr:",result);
```

```
// 5. short:-
```

//problem???: because it sort in behafe of aski values.

```
// var arr=[1,2,11,4,5,33,3];
```

```
// const getResult=arr.sort();
```

```
// console.log(getResult);
```

//solution:--

```
// var arr=[1,2,11,4,5,33,3];
```

```
// const getResult=arr.sort((a,b)=>{
```

```
//   // // in this a refer to first number in array and b reffer to last.
```

```
//   // // assendeing order:
```

```
//   return a-b;
```

```
// })
```

```
// console.log(getResult);
```

```
// //// deccendeing order:
```

```
// var arr=[1,2,11,4,5,33,3];
```

```
// const getResult=arr.sort((a,b)=>{
```

```
//   return b-a;
```

```
// });
```

```
// console.log(getResult);
```

//6. find :---

/////find the first accurance of the element.

```
// var arr=[2,2,1,4,1,5];
```

```
// const findResult=arr.find((item,index,arr)=>{
```

```
//   return item == 1;
```

```
// })
```

```
// console.log(findResult);
```

```
//findIndex: used to find the index..
```

```
// var arr=[2,1,3,4,1,5];
```

```
// const findResult=arr.findIndex((item,index,arr)=>{
```

```
//   return item == 1;
```

```
// });
```

```
// console.log(findResult);
```

```
//=====
```

```
=====
```

```
//some :
```

```
//it takes a call by function as a input and within a call by function it
```

```
// takes a parameter and returns a bollean data.
```

```
// const arr=[2,3,4,5,6,7];
```

```
// const getResult=arr.some((item,index,arr)=>{
```

```
// return item %2 ==0; //check some item is devide by 2
```

```
// })
```

```
// console.log(getResult);
```

```
//=====
```

```
=====
```

```
//every:-
```

```
// const arr=[2,3,4,5,6,7];
```

```
// const getResult=arr.every((item,index,arr)=>{
```

```
// return item %2 ==0; //check every item is devide by 2
```

```
// })
```

```
// console.log(getResult);
```

```
//=====
```

```
=====
```

//fill method:--

// const getarray=new Array(5); /////this is another method to declare array.with "new" keyword.

// //getarray.fill(2);///// this method is used to fill all empty element in array with 2.

// getarray.fill(2,1,3);/////in this fill method it indicates that 2 is filled between

// ////empty area b/w ,,,1&2 index

// console.log(getarray);

//=====

// //slice:

// slice method is used to give a new selected array and does not cahnge the

// // original array

// const arr=[1,2,3,4,5];

// const sliceResult=arr.slice(0,3);// // here 0 and 2 are the index.

// // // display the data b/w 0 and 2 indexes.

// console.log(sliceResult);

//=====

//splice method:

// //1.delete data in b/w array.

// const arr=[1,2,3,4,5];

// arr.splice(1,2);/////1 and 2are index used to delete data on 1&2.

// console.log(arr);

// //2. insert data:

```
// const arr=[1,2,3,4,5];
// arr.splice(1,2,"item1","item2");
// // // item1 & item2 is inserted inplace of
// // // deleted data.
// console.log(arr);
//=====
=====
```

```
//=====
=====
```

```
//set method:
// it is a object with collection of values and it contain a unique value.
// and you can also add array in it .
```

```
// const arr=[1,2,3,3,4,3];
// const result=new Set(arr);
// console.log(result);
// // add in set =====
```

```
//case 1--
```

```
// const arr=[1,2,3,3,4,3];
// const result=new Set(arr);
// result.add(5);
// result.add(5);
// console.log(result);
```

```
// case 2---
```

```
// // a>
```

```
// const arr=[1,2,3,3,4,3];
```

```
// const result= new Set();
```

```
// result.add(arr);
```

```
// result.add(arr);
```

```
// console.log(result);
```

```
// // b>
```

```
// const arr=[1,2,3,3,4,3];
```

```
// const arr2=[1,2,3,3,4,3];
```

```
// const result= new Set();
```

```
// result.add(arr);
```

```
// result.add(arr2);
```

```
// console.log(result);
```

```
// // //has in set: ---
```

```
// const arr=[1,2,3,3,4,3];
```

```
// const result= new Set();
```

```
// result.add(arr);
```

```
// result.add(10);
```

```
// result.add(11);
```

```
// result.add("dhruv");
```

```
// console.log(result);
```

```
// console.log(result.has(10));
```

```
//=====
```

```
=====
```

```
// currying in javaScript
```

```
// Currying is a concept in functional programming where a
```

```
// function that takes multiple arguments is transformed into
// a sequence of functions, each accepting a single argument
// does not call a function just transform it.
```

```
// function fun1(l){
//   return function fun2(b){
//     return function fun3(h){
//       return l*b*h ;
//     }
//   }
// }
```

```
// const obj= fun1(1) (2) (3);
// console.log( "volume of rectangle = ", obj);
```

```
//ex:2
```

```
// function double(a){
//   return function(b){
//     return a*b;
```

```
// }
```

```
// }
```

```
// const double2 = double(2); //pass to a value
```

```
// console.log(double2(3)); // pass to b
// console.log(double2(5));
// //above a remain constant & b can be change using currying
//
```

```
=====
=====
```

```
// array methods :==
// //1.concat:
// const a=["a","b","c","d"];
// const result=a.concat(["e"]);
// console.log(result);
// //
// const a=["a","b","c","d"];
// const result=[].concat(a);
// console.log(result);
//2. join
// const a=["a","b","c","d"];
// const result=a.join("-") ;// it can be ~ etc.
// // const result=a ;//object
// console.log(result);
// console.log(typeof result);//string
// //3.slice
// const a=["a","b","c","d"];
// const result=a.slice(1,2);
// console.log(result);
// //4.indexOf
```

```
// const a=["a","b","c","d"];
// const result=a.indexOf("a");
// console.log(result);
//5. lastIndexOf
// const a=["a","b","c","d","a"];
// const result=a.lastIndexOf("a");
// console.log(result);
// // 6. at:
// const a=["a","b","c","d"];
// const result=a.at(1);
// console.log(result);
// //7.pop:
// const a=["a","b","c","d"];
// const result=a.pop();
// console.log(result);
// console.log(a);
// // 8.push:
// const a=["a","b","c","d"];
// const result=a.push("xxx");
// console.log(result);
// console.log(a);
// //9.unshift:
// const a=["a","b","c","d"];
// const result=a.unshift("xxx");
// console.log(result);
```

```
// console.log(a);

//10. shift:

// const a=["a","b","c","d"];

// const result=a.shift();

// console.log(result);

// console.log(a);

// //11. fill:

// const get =new Array(5);

// get.fill(1);

// get.fill(1,2);//fill "1" except in first two indexes.

// console.log(get);

//12. reverse:

// const a=["a","b","c","d"];

// const result=a.reverse();

// console.log(result);

// console.log(a);

//13.includes: it gives us boolean value that data is present or not.

// const a=["a","b","c","d"];

// const result=a.includes("a");

// console.log(result);

// console.log(a);

// //14.find:

// const a=["a","b","c"];

// const get=a.find((item,index,arr)=>{

//   return item=="a";
```

```

// });
// console.log(get);
// //15.findIndex:
// const a=["a","b","c"];
// const get=a.findIndex((item,index,arr)=>{
//   return item=="a";
// });
// console.log(get);
//16. forEach
//17. map
//18. filter
//19. reduce
//=====
=====

// //window:
// console.log("window",window);
// //this:
// console.log("this",this);
// console.log(window===this);
//=====
=====

// ##### window object:==
//it is a global object ,All things of JavaScript like function ,array etc...
// are kept inside this window object .
// this object:=====
// "this object" is a global object that refer to window object.

```

```
//console.log(this);
```

// // Q > prove that whatever you write in javaScript will stored in window.??

```
// function myFun(){
```

```
//   console.log("hello my fun ..is workeing!");
```

```
// }
```

```
// window.myFun();
```

```
// myFun();////by default the window is attach to it but not visible.
```

```
// ///// ex:-   window.fun
```

```
// console.log(window);
```

```
//=====
```

```
// NOTE: inside the function "this object" call the global object window.
```

```
// function myFun(){
```

```
//   console.log(this);
```

```
// }
```

```
// myFun();
```

```
// ### "use strict" :
```

// With the use of "use strict" key word ,,"this object" will refer to undefined.

```
//it always declared top of the module/ code.
```

```
// function myFun(){
```

```
//   "use strict"
```

```
//   console.log(this);
```

```
// }
```

```
// myFun();
```

```
// Q. how to use "this " praactically ??
```

```

// const studentInfo2={
//   studentAddress:"katwaria sarai",
// }

// const studentInfo = {
//   studentName: "dhruv",
//   studentID:"123",
//   studentEmail:"ronack@gmail.com",
//   getinfo:function()  //// or you can call like this getinfo(){}
//   {////insted of useing object name "studentInfo" again and again you
can use ,,
//   ////console.log(studentInfo.studentName,studentInfo.studentID);
//   //// you can use "this" to access your own property,,
//   console.log(this.studentName,this.studentID);
//   ////console.log(this.studentAddress);////you can not call outside
property with "this".

//   ////you have to use name of the outside object :=
//   console.log(studentInfo2.studentAddress);
// }
// }

//   studentInfo.getinfo();

//// Q> how to call your own data/property inside function??
//// 1. to get the whole data.
// function myfun(){
//   this.name="dhruv" ;
//   this.sem="6th" ;

```

```
// console.log(this);
```

```
// }
```

// new myfun(); // with the use of new keyword you can access "this" in function.

```
////2.to get the paticular data.
```

```
// function myfun(){
```

```
//   this.name="dhruv";
```

```
//   this.sem="6th";
```

```
// return this;
```

```
// }
```

```
// const data= new myfun;
```

```
// console.log(data.name);
```

```
// Q> use "this " in side the inner function to get the data.
```

```
// const objInfo={
```

```
//   name:"dhruv",
```

```
//   getInfo:function(){
```

```
//     innerfunction :function() {
```

```
//       console.log(this.name);
```

```
//     }
```

```
//     innerfunction();
```

```
//   }
```

```
// }
```

```
// objInfo.getInfo();
```

```
//in this inner function is not refer to the obj info ,,it is refering to the
```

//global object thats why you are getteing result undefined.. to resolve this

//issue you can use arrow function in your inner function

```
// const objInfo={
```

```
//   name:"dhruv",
```

```
//   getInfo:function(){
```

```
//     const innerfunction= ()=>{
```

```
//       console.log(this.name);
```

```
//     }
```

```
//     innerfunction();
```

```
//   }
```

```
// }
```

```
// objInfo.getInfo();
```

```
//=====
```

```
=====
```

//### call ,apply and bind method in javaScript.

//1.call: it execute the function with provided context and parameter,

//call method takes the arguments seprat

// //to call object data from inside another object function we can use call method.

// Q.> calling function using "call method" and by using context only?

```
// //wat1:
```

```
// const studentRecord={
```

```
//   name:"dhruv",
```

```
//   class:"12TH"
```

```
// }
```

```

// const studentInfo={
//   getInfo:function(){
//     return `${this.name} ${this.class}`
//   }}

//problem

// const r=studentInfo.getInfo;
// console.log(r); //it will return the whole function
// console.log(studentInfo.getInfo);//it will return the whole function

//solution-

// const result= studentInfo.getInfo.call(studentRecord)
// console.log(result);

//way 2: without return-

// const studentRecord={
//   name:"dhruv",
//   class:"12TH"
// }

// const studentInfo={
//   getInfo:function(){
//     console.log(`${this.name} ${this.class}`);
//   }}

//   studentInfo.getInfo()

// studentInfo.getInfo.call(studentRecord)

```

// Q.> calling function using "call method" and by using both context and parameter

// only?

```
// const studentRecord={
//   name:"dhruv",
//   class:"12TH"   // name & class are context
// }
// const studentInfo={
//   getInfo:function(course){   // course is parameter.
//     console.log(`${this.name} ${this.class} ${course}`);
//   }}
//   studentInfo.getInfo.call(studentRecord,"arts")
//   console.log(studentRecord);
// //=====
```

//2. APPLY : it execute the function with provided context and but parameter

//as an array .

//NOTE: apply method work with context just like call ex:-

```
// const studentRecord={
//   name:"dhruv",
//   class:"12TH"   // //context
// }
// const studentInfo={
//   getInfo:function(){
```

```

//    return `${this.name} ${this.class}`
//  }}
//  const result= studentInfo.getInfo.apply(studentRecord)
// console.log(result);
// NOTE : but,,it take parameter as a an array ex:-
//  const studentRecord={
//    name:"dhruv",
//    class:"12TH"    // name & class are context
//  }
//  const studentInfo={
//    getInfo:function(course,address,number){    // course is parameter.
//      console.log(`${this.name} ${this.class} ${course} ${address}
//        ${number}`);
//    }}
//    studentInfo.getInfo.apply(studentRecord,["arts","katwaria","9911xx"])
;

//=====

```

//3.bind (IMP) : it bind the function with provided value and context but it does

//not execute the function. to execute the function we should call the function,

//case 1: how tp call bind method

```

//  const studentRecord={
//    name:"dhruv",
//    class:"12TH"    // name & class are context
//  }

```

```

// const studentInfo={
//   getInfo:function(course,address,number){ // course is parameter.
//     console.log(`${this.name} ${this.class} ${course} ${address}
//       ${number}`);
//   }}
// studentInfo.getInfo.bind(studentRecord,["arts","katwaria","9911xx"])
// ()
//case2: with the use of bind method object can borrow a method from
// //another object.
// const studentRecord={
//   name:"dhruv",
//   class:"12TH"
// }
// const studentInfo={
//   getInfo:function(){
//     return `${this.name} ${this.class}`
//   }}
// const result= studentInfo.getInfo.bind(studentRecord)
// console.log(result());
//case 3: this loss condition

// const studentInfo={
//   name:"ronack",
//   getInfo:function(){
//     return `${this.name} ${this.class}`

```

```
//  }}  
//  const result= studentInfo.getInfo;  
// console.log(result());
```

```
/// / preserving this loss condition:
```

```
//  const studentInfo={  
//    name:"ronack",  
//    getInfo:function(){  
//      return `${this.name}`  
//    }}  
//  const result= studentInfo.getInfo.bind(studentInfo);  
// console.log(result());
```

```
//  const result= studentInfo.getInfo.bind(studentInfo);  
// console.log(result());
```

```
//=====
```

```
=====
```

```
//lexical scope in javaScript:--
```

```
//lexical scope define the accessability of variables and function
```

```
//depend upon there location in above source code
```

```
// ex:-
```

```
//case 1:
```

```
// function outer(){
```

```
//  const sname="dhruv singh"; // lexical scope for inner function.
```

```
//    function inner(){
```

```
//  //  const sname="ronack singh"
```

```
//    console.log(sname);
```

```
//  }
```

```
// inner();
// }
// outer(); //// inside outer function ,inner function will print above sname
// ////which is its lexical area.
//=====
// CASE 2:
// const sname="dhruv singh"; // lexical scope for outer function.
```

```
// function outer(){
//           // lexical scope for inner function.
//   function inner(){
//     // //const sname="ronack singh"
//     console.log(sname);
//   }
//   inner();
// }
```

```
// outer(); //// inside outer function ,,,
//inner function will check its lexical scope area //and not find anything
//now outer scope will check its lexical scope area and print sanme..
```

```
//=====
```

```
=====
```

```
//object. create():---
// // it is used to create a variable (i.e let etc.) ,to a object .
// const obj = {
//   color1:"red ",
```

```
// color2:"blue",
// get : function(){
//   console.log(`${this.color1} ${this.color2}`)
// }
// }
// const result=obj.get();
// // console.log(result);//undefined
// console.log(typeof result);
// it will give undefined.
```

```
// // // how to mAKE result/variable as object ???
// const result=Object.create(obj);
// console.log(result);
// console.log(typeof result);
//
```

```
=====
=====
```

```
//oprations on object:
//case 1: rename: it used to change the key value of an object.
// const obj={
//   name:"dhruv"
// }
// const result=obj.name="ronack";
// console.log(result);
// console.log(obj)
//
```

```
// // case2: add:
// const obj={
//   name:"dhruv"
// }
// obj.BLOODgroup="a+";
// console.log(obj);
// //case3: delete
// const obj={
//   name:"dhruv",
//   sname:"singh"
// }
// delete obj.name;
// console.log(obj);
```

```
//=====
```

==

```
// object.freeze();
```

// freezeing an object doesnot allow new properties to be added to an object

//and prevents revomeing & altereing the existing properties of an Object.

```
// //ex:-
```

```
// const obj={
//   name:"dhruv",
//   sname:"singh"
// }
```

```
// Object.freeze(obj)
```

```
// delete obj.name;
```

```
// console.log(obj);
```

```
//=====
```

```
=====
```

```
//class and the constructor:
```

```
//classes are the templates used for createing the object.
```

```
//object are instance of the class
```

```
// class are the blue print for the object.
```

```
//Q> how to initialize class and its methods??
```

```
// class abc{
```

```
//   constructor(){ // by default class contain constructor..
```

```
//   console.log("hello iam a constructor .");
```

```
// }
```

```
//   method1(){
```

```
//     console.log("mwthod 1 from class abc.");
```

```
// }
```

```
//   method2(){
```

```
//     console.log("mwthod 2 from class abc.");
```

```
// }
```

```
//}
```

```
// new abc;    // this is how we call a constructor.
```

```
// //=====
```

```
// // how to call methods in class:
```

```
// const obj= new abc;
```

```
// obj.method1();
```

```
//=====
```

```
// //Q> how to pass arguments to class?
```

```
// class abc{
```

```
//   constructor(item1,item2){ // item 1 and item2 are parameters.
```

```
//   console.log("hello iam a constructor .");
```

```
// }
```

```
//   method1(){
```

```
//     console.log("method 1 from class abc.");
```

```
// }
```

```
//   method2(){
```

```
//     console.log("method 1 from class abc.");
```

```
// }
```

```
// }
```

// const obj= new abc ("apple","milk"); // apple and milk are the arguments passed to the parameters.

```
// obj.method1();
```

```
// =====
```

```
//Q> how can access the data from the constructor ??
```

```
//Q> how to pass arguments to class?
```

```
// class abc{  
//   constructor(item1,item2){ // item 1 and item2 are parameters.  
//     console.log("hello iam a constructor .");  
//     this.a=item1;  
//     this.item2=item2; // you can take any name istem of item2 with  
this.ex:- this.a !
```

```
//   }
```

```
//   method1(){  
//     console.log("mwthod 1 from class abc.");  
//     console.log(this.a);  
//   }
```

```
//   method2(){  
//     console.log("mwthod 2 from class abc.");  
//     console.log(this.item2);
```

```
//   }
```

```
// }
```

```
// const obj= new abc ("apple","milk"); // apple an dmilk are the  
arguments passed to the parameters.
```

```
// obj.method1();
```

```
// obj.method2();
```

```
//=====
```

```
=====
```

```
//extends keywords:
```

// used to extend property of parent/super class into child/sub class.

```
// class parent{  
// constructor()  
//     { console.log("parent constructor");}
```

```
//     method(getItem)  
// {console.log("parent method",getItem);}  
// }
```

```
// class child extends parent{
```

```
// };
```

// Q> how to call parent constructor with child class??

```
// new child;
```

//Q> how to access parent class methods with the child class??

```
    // const obj= new child;
```

```
    // obj.method();
```

//Q> how to pass argumts to the methods??

```
//obj.method("ITEM");
```

```
//=====
```

//Q> how to pass arguments to method by using the constructor??

```
//solv:_
```

```

// class parent{
// constructor(getItem)
//     { console.log("parent constructor");
//       this.getItem=getItem;
//     }

//     method()
// {console.log("parent method,,,",this.getItem);}
// }

// class child extends parent{
// childMethod(){
// console.log("child method,,",this.getItem);
// }
// };

// // using child class object passed the value to the parent constructor.
// const obj=new child("item1");
// obj.method();

// //acces parent constructor with child object..
// obj.childMethod();
// //=====

//Q. use of super keyword in child class?

// class parent{
// constructor(getItem)
//     { console.log("parent constructor");

```

```
//    this.getItem=getItem;
```

```
//    }
```

```
//    method()
```

```
// {console.log("parent method,,,",this.getItem);} 
```

```
// }
```

```
// //==
```

```
// class child extends parent{
```

```
// // to use constructor in child class // you have to use super() keyword.
```

```
// //way1:
```

```
// //  constructor(getItem)
```

```
// //    {  super();
```

```
// //        console.log("child constructor");
```

```
// //        this.getItem=getItem;
```

```
// //    }
```

```
// //way 2:
```

```
// constructor(getItem)
```

```
//    {  super( getItem);
```

```
//        console.log("child constructor");
```

```
//    }
```

```
// }
```

```
// const obj=new child("item1");
```

```

// obj.method();

//=====
=====

// Q>how to remove duplicate data from the array??

// way1: useing method.

// const arr=[1,2,3,4,5,1,2,3,4,5];

// const result=new Set(arr);

// console.log(result);

// way2:

// const arr=[1,2,3,4,5,1,2,3,4,5];

// const arr2= [];

// //console.log(arr.indexOf(6)); // as we know -1 because 6 is not in array

// for(var i=0;i<arr.length;i++)

// {

//     // if(arr2.indexOf(arr[i])==-1)

//     // {arr2.push(arr[i])}

//     }

// console.log(arr2);

//

//=====
=

//get missing number //or// insert missing number

```

```
// const arr=[1,3,5,7,8];
```

```
// const newArr =[];
```

```
// for(i=1;i<=8;i++)
```

```
// {console.log(i);
```

```
//   if(arr.indexOf(i)==-1)
```

```
//   {newArr.push(i);}
```

```
// }
```

```
//   console.log("missing in array:",newArr);
```

```
//=====
```

```
=====
```

```
//counting words in string.
```

```
// const sname= "dhruvd singh";
```

```
// ////here split perform where space is not there
```

```
// const splitResult=sname.split(""); ////"" inndicates space
```

```
// console.log(splitResult);
```

```
// const count=[];
```

```
// for(arrItem of splitResult){
```

```
//   // console.log(arrItem);
```

```
//   // count[arrItem]=1;
```

```
//   count[arrItem]=count[arrItem]?count[arrItem]+1:1 ;
```

```
// }
```

```
// console.log(count);
```

```
//=====
=====
```

```
//reverse the data in array:
```

```
//way1:
```

```
// const arr=[1,2,3,4,5];
```

```
// // console.log(arr.length)
```

```
// const rev=[];
```

```
// for(i=arr.length-1;i>=0;i--){
```

```
//     //console.log(i);
```

```
//   rev.push(arr[i]);
```

```
// }
```

```
// // console.log(rev);
```

```
// //way 2:
```

```
// const arr=[1,2,3,4,5];
```

```
// const result=arr.sort((a,b)=>{
```

```
// // console.log(a); //remove first element.
```

```
// // console.log(b); //remove last element.
```

```
// if(a>b) return -1;
```

```
// })
```

```
// console.log(arr);
```

```
//=====
=====
```

```
//fibonaic series:
```

```
// let n1=0;
```

```
// let n2=1;
```

```
// let n3 ;
```

```

// let max=5;

// for(let i=0;i<max;i++){
//   console.log(n1);
//   n3=n1+n2;
//   n1=n2;
//   n2=n3;
// }

//=====

// Q> how many time a string is in another string??

// data1="d";
// data2="dhruv singh";
// let count=0;
// function fun(getdata1,getdata2)
// {
//   for(i=0;i<getdata2.length;i++){
//     // console.log( getdata2.charAt(i));
//     if(getdata2.charAt(i)==getdata1)
//       {count += 1;}
//   }
//   return count;
// }

// const result=fun(data1,data2);
// console.log("repeateing no.of times:" , result);

```

```
//=====
=====
```

```
//swap two numbers:
```

```
// //way1:
```

```
// var a= 1;
```

```
// var b=2;
```

```
// var [b,a]=[a,b];
```

```
// console.log("a:",a);
```

```
// console.log("b",b);
```

```
// //way2:
```

```
// var a=1;
```

```
// var b=2;
```

```
// let temp=a;
```

```
// a=b;
```

```
// b=temp;
```

```
// console.log("a:",a);
```

```
// console.log("b",b);
```

```
//=====
```

```
//how to fix array lenght
```

```
// const arr=[1,2,3,4,5,6];
```

```
// // console.log(arr.length); ///// 6
```

```
// arr.length=4;
```

```
// console.log(arr); //till 3 index answer
```

```
//=====
```

```
//Closure function
```

// In JavaScript a closure is a function that remembers the environment in which it was created even after the outer function has finished executing

```
// In below code:
```

// With the use of closure function (i.e counter) you can access private variable (i.e count) outside function.

```
// A closure happens only when:
```

// A function remembers and uses variables from its outer (parent) function

```
// Even after the outer function has finished executing
```

```
// function counter() {
```

```
//   let count = 0;
```

```
//   return {
```

```
//     increment: function () {
```

```
//       count++;
```

```
//       return count;
```

```
// },
```

```
//   decrement: function () {
```

```
//     count--;
```

```
//     return count;
```

```
//   },
```

```
//   displayCount: function () {
```

```
//     let message = "Current count: " + count;
```

```
//     return message;
```

```
//   }
```

```
// };
```

```
// }
```

```
// const myCounter = counter();
```

```
// // console.log(myCounter);
```

```
// console.log(myCounter.displayCount());
```

```
// console.log(myCounter.increment());
```

```
// console.log(myCounter.displayCount());
```

```
//=====
```

```
// setTimeout
```

```
// const timerId = setTimeout(() => {
```

```
// console.log("Hello after 2seconds");
```

```
// }, 2000);
```

```
// clearInterval(timerId)
```

```
// setInterval
```

```
// setInterval(() => {
```

```
// console.log("Hello after every 2 seconds");
```

```
// }, 2000);
```

```
// setInterval & set timeout
```

```
// const timerId = setInterval(() => {
```

```
// console.log("Hello after every 2 seconds");
```

```
// }, 2000);
```

```
// setTimeout(() => {
```

```
// clearInterval(timerId)
```

```
// }, 4000);
```

```
//=====
```

```
//promises:
```

// Promises handle asynchronous tasks in JavaScript by providing a more readable

// and structured approach than callbacks for handling outputs

// const data ={name: "John", age: 30 };

// function fetchData(){

// return new Promise((resolve, reject) => {

// setTimeout(() => {

// resolve(data);

// }, 3000); // 3 sconds

// })

// }

// //var obj=fetchData();

// //console.log(obj); //promise

// fetchData().then(dataObj => {

// console.log("Data: ", dataObj);

// }).catch(err => {

// console.log("Error: ", err);

// })

// /=====

```
//async & await

// async function fetchData() {

    // try{

        // const response = await
fetch("https://jsonplaceholder.typicode.com/todos/1");


        // const data = await response.json();
        // console.log("data fetched:",data);


    // } catch(err){
    // console.log("Error: ", err);
    // }
    // }

// fetchData();

//=====

// generator function


// function* infiniteSequence(){
//   let num=1;
//   while(true){
//     yield num;
//     num++;
//   }
}
```

```
// }
```

```
// const seq = infiniteSequence();
```

```
// console.log(seq.next().value);
```

```
// console.log(seq.next().value);
```

```
//=====
```

```
//weak map and weak set:
```

```
//weak map:
```

```
// let weakMap = new WeakMap();
```

```
// let obj = { name: "Adam" };
```

```
// weakMap.set(obj, "Employee");
```

```
// // obj = null; //use to remove data in obj!
```

```
// // console.log(weakMap);
```

```
// console.log(weakMap.get(obj)); //employee
```

```
// // console.log(weakMap.has(obj));
```

```
//weak set:
```

```
// let weakSet = new WeakSet();
```

```
// let obj = { name: "Adam" };
```

```
// weakSet.add(obj, "Employee");
```

```
// // obj = null; //use to remove data in obj!
```

```
// console.log(weakSet.has(obj)); //true
```

```
///=====
```

```
// Shallow Copy
```

```
// const original =
```

```
// {
```

```
// name: "John",
```

```
// address: {
```

```
// city: "Delhi" }
```

```
// };
```

```
// //way 1:Object.assign
```

```
// const shallowCopy = Object.assign({}, original);
```

```
// // console.log(shallowCopy);
```

```
// // way 2: Spread Operator
```

```
// const shallowCopy2 = { ... original};
```

```
// shallowCopy2.address.city = "New city";
```

```
// console.log(original.address.city);//new city
```

```
//deep copy
```

```
// const original = {
```

```
//   name: "John",
```

```
//   address: { city: "Delhi" }
```

```
// };
```

```
// //json .parse covert it to object
```

```
// const deepCopy = JSON.parse(JSON.stringify(original)); //it will convert  
the object into the string
```

```
// console.log(deepCopy);
```

```
// deepCopy.address.city="new city";
```

```
// console.log(deepCopy.address.city);  
// console.log(original.address.city);
```

(HTML)

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
<title>java_script</title>  
<script>  
    // console.log("hello world");  
    // console.warn("this is used for warning");  
    // console.error("this is used for error");  
</script>  
</head>  
<body>  
    <script>  
        // console.log("hello world");  
        // console.warn("this is used for warning.");  
        // console.error("this is used for error .");  
    </script>
```

```
<script src="./class1.js"></script>
```

```
<!-- //===== --
```

```
>
```

```
<!-- -->
```

```
<!-- //topic 1: Dom communication. //sep 22 video on saved messages on  
telegram -->
```

```
<!-- // DOM stands for Document Object Model. It is a programming  
// interface that represents the structure of a web page as a hierarchical  
// tree of objects. This model allows programming languages like  
JavaScript
```

```
// to dynamically access, modify, and interact
```

```
// with the content, structure, and style of a web document.
```

```
// The DOM is essential for creating dynamic and interactive web  
applications.
```

```
// It bridges the gap between static HTML/CSS and dynamic JavaScript  
functionality.
```

```
// For example, JavaScript can use the DOM to change HTML elements,  
update styles,
```

```
// handle user interactions, and even create or remove elements in real-  
time.
```

```
// Key Features of the DOM----
```

```
// Tree Structure: The DOM represents the HTML document as a tree,
```

// where each element, attribute, and text is a node. For example,
// the <html> tag is the root node, and its children include <head> and
<body>.

// Dynamic Interaction: It allows developers to update content,
// styles, and structure without reloading the page. For instance,
// you can change the text of a paragraph or add new elements
dynamically.

// Event Handling: The DOM supports event handling, enabling
// actions like responding to button clicks or form submissions.

// Cross-Platform: It is platform-independent and works across different
// browsers and programming languages.

//as we know web page created using html elements

//whenever a web page is loaded in browser then, browser create dom.

//dom structure is created in form of tree of objects. -->

```
<!-- <div id ="xyz">
```

```
hello from div1 and id :xyz.
```

```
</div> -->
```

```
<!-- <div class ="xyz">
```

```
hello from div2 and class :xyz.
```

```
</div>
```

```
<div class ="xyz">
```

hello from div3 and class :xyz.

</div> -->

<script>

//Q> how to get data of div by in javaScript?

//solv: using "getElementById" method ex:-

// //here document is object we can acces object data using key name which is xyz i.e is div id name.

// const getdata=document.getElementById('xyz');

// console.log(typeof getdata); //object

// console.log(getdata); // it will return the whole div

//=====

// what if the div's class have same names ?

//solv:it will give in array form.

// const getdata2=document.getElementsByClassName('xyz');

// console.log(getdata2);

//=====

// Query selector : Query Selector in JavaScript

// The querySelector method in JavaScript is used to select the

// first element that matches a specified CSS selector.

// It is a powerful tool for accessing and manipulating DOM elements.

// //for id ,,ex:-

// const getData3=document.querySelector('#xyz'); ///// # used to
idencitate id.

// console.log(getData3);

//=====

// note: querySelector alway refer to first prefrence: ex--

//for class ,,ex:-

//there are two class anmed xyz but it will give prefrence to the first class.

// const getData4=document.querySelector('.xyz'); // "." used to
idencitate class.

// console.log(getData4);

//=====

// // how to get inner text

// const getData=document.getElementById('xyz').innerText;

// console.log(getData);

//=====

// how to update text in div:

// using innerHTML tag.

// const getData=document.getElementById('xyz').innerHTML="text
changed in div with id: xyz";

// console.log(getData);

//=====

// // tag keyword:

// const getData=document.getElementsByTagName('div');

// console.log(getData);

```
//
</script>
<!-- ////////////////////////////////////// -->
<!-- topic:2 -->
<!-- java script events: -->
<!-- <script>
function fun(){
    console.log("iam afunction");
}

function fun2(item){
    console.log("iam a function. ",item);}

function fun3(item,item2){
    console.log("iam a function. ",item,item2);
}

</script> -->
<!-- <button onclick ="fun()"> click </button> -->
<!-- <button ondblclick ="fun()"> click </button>
<button onmousemove ="fun()"> click </button>
<button onmouseout ="fun()"> click </button>
<button onmouseenter ="fun()"> click </button> -->
<!-- //===== -->
```

```
<!-- <input type="text" placeholder="
enter" onchange="fun2(this.value)"> -->
```

```
<!-- <input type="text" placeholder="
enter" onkeypress="fun2(this.value)"> -->
```

```
<!-- <input type="text" placeholder=" enter" onkeyup="fun2(this.value)">
-->
```

```
<!-- argument and parameter -->
```

```
<!-- <button onclick="fun3('hello','world')"> click</button> -->
```

```
<!-- //promise -->
```

```
<script>
```

```
// let myPromise = new Promise(function(resolve, reject) {
// let success = true; // Change this to false to simulate an error
// if (success) {
// resolve("The data has loaded successfully!");
// } else {
// reject("There was an error loading the data.");
// }
// });
```

// // To handle the result of a Promise, you use the then and catch methods:

```
// myPromise.then(function(message) {
// console.log(message); // This runs if the promise is fulfilled
// }).catch(function(error) {
// console.log(error); // This runs if the promise is rejected
// });
</script>
```

<!-- topic: EVENT Propagation -->

<!-- phase1: Event Bubbling.

in below code is to show that when we click on button it will propagate to
div

it will continue to propagate..

-->

<!-- <div> -->

<!-- //we did not add any Event (i.e onClick etc..) to button -->

<!-- <button >submit</button>

link ex to stop

</div> -->

<script>

```
// var div=document. querySelector("div");
```

```
// var button=document.querySelector("button");
```

```
// var a=document.querySelector("a");
```

```
// phase1: Event Bubbling
```

```
// div.addEventListener("click",()=>{
```

```
//    // 'DIV' : div got its Event listener which we now apply to its content like  
button
```

```
// console.log("div")
```

```
// })
```

```
// button. addEventListener("click",()=>{
```

```
// console.log("button")
```

```
// })
```

```
// Phase 2:Event Capturing
```

```
//by adding true it will reverse the order of event bubbling propagation.
```

```
// div.addEventListener("click",()=>{
```

```
//    // 'DIV' : div got its Event listener which we now apply to its content like  
button
```

```
// console.log("div")
```

```
// },true)
```

```
// button. addEventListener("click",()=>{
```

```
// console.log("button")
```

```
// },true)
```

```
//topic 3: stop propagations: of Event bubbling !
```

```
// button.addEventListener("click",(event)=>{
```

```
// event. stopPropagation();
```

```
// console.log("button"))}
```

```
// topic 4: stop default behaviour of element
```

```
//a.addEventListener("click",(event)=>{
```

```
//event.preventDefault();
```

```
//console.log("href stops")
```

```
//})
```

```
</script>
```

```
<!-- <p>click on inner div text below :</p>
```

```
<div id="topDiv">
```

```
  "topDiv click"
```

```
  <div id="midDiv">
```

```
    "mid div clicked"
```

```
    <div id="innerDiv">
```

```
      "innerDiv clicked"
```

```
    </div>
```

```
  </div>
```

```
</div> -->
```

```
<script>

// document.getElementById("topDiv").addEventListener
// ("click", () => {
// console.log("Outer div has been clicked");
// }, true)

// document.getElementById("midDiv").addEventListener
// ("click", () => {
// console.log("Mid div has been clicked");
// }, true)

// document.getElementById("innerDiv").addEventListener
// ("click", () =>{
// console.log("Inner div has been clicked");
// },true);

</script>
```

```
<!-- topic: What is event delegation -->
```

```
<!-- <div className="box">
<ul id="itemList">
<li>Item 1</li>
<li>Item 2</li>
<li>Item 3</li>
```


</div> -->

<script>

//without Event diliation !

// const items = document.querySelectorAll('#itemList li');

// items.forEach(item => {

// item.addEventListener("click", function () {

// console.log("Clicked: ", this.textContent);

// })

// })

//with Event diliation !

// const items = document.getElementById('itemList');

// items.addEventListener("click", function(e){

// if(e.target.tagName === "LI"){

// console.log("Clicked: ", e.target.textContent);

// }

// })

</script>

```
<!-- <div id="divx">
```

```
  <button id="buttonx">
```

```
  </button>
```

```
  <a id="ahref" href="./indexnew.html">helo</a>
```

```
</div> -->
```

```
<script>
```

```
// let div=document.getElementById("divx");
```

```
// let button=document.getElementById("buttonx");
```

```
// let a=document.getElementById("ahref");
```

```
// div.addEventListener("click",()=>{
```

```
//   console.log("div clixked");
```

```
// })
```

```
// a.addEventListener("click",(event)=>{
```

```
//   // event.stopPropagation();
```

```
//   event.preventDefault();
```

```
//   console.log("bttton clixked");
```

```
// })
```

```
</script>
```

```
<ul id="ulid">
```

```
  <li>dhruv</li>
```

```
<li>pritish</li>
```

```
</ul>
```

```
<script>
```

```
let items=document.getElementById("ulid");
```

```
items.addEventListener("click",function(e){
```

```
if(e.target.tagName === "LI"){
```

```
  console.log(e.target.textContent);
```

```
}
```

```
})
```

```
</script>
```

```
</body>
```

```
</html>
```

